

Lab Procedure for Python Sensor Fusion

Setup

1. It is recommended that you review [Lab 2 – Application Guide](#) before starting this lab.
2. The code for this lab builds on **Lab 1: Localization** in Skills Progression 2. Ensure that the relevant code sections are completed with your code from Lab 1:
 - a. In Section B, right-click on **QBPRanging()** and select "go to definition" to view the class definition within the `qbot_platform_functions` module. Complete the functions **adjust_and_subsample()** and **correct_lidar()**. In the same file, scroll to the **QBPLocalization** class and complete the functions **save_ref()** and **scan_match()**.
 - b. In `sensor_fusion.py`, complete the code in Sections D and E to read and correct raw LiDAR data, save a reference scan, and set the **refCollected** flag.
 - c. Modify the line that calls the **scan_match()** function to pass in the corresponding parameter values from Lab 1.
3. Launch Quanser Interactive Labs, scroll to the "QBot Platform" menu item and then select the "Warehouse" world.

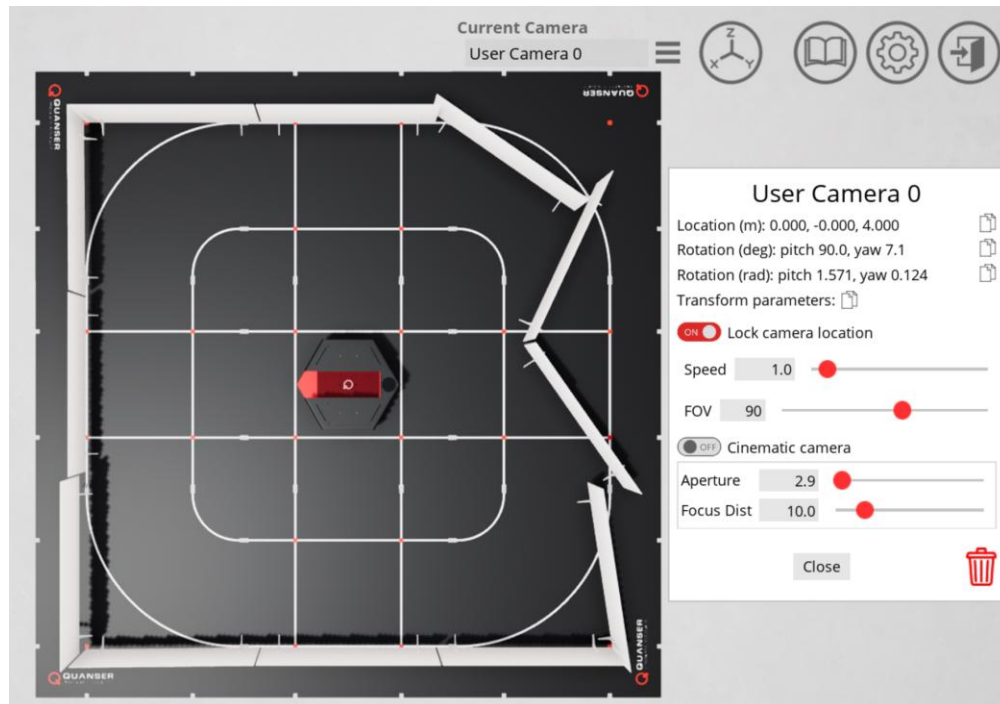


Figure 1: Successful set up of the Quanser Interactive Labs Workspace

4. Open and run the script [sensor_fusion.py](#). Verify that the script ran successfully by checking that the QBot and other elements have been spawned within the virtual environment in Quanser Interactive Labs, as shown in Figure 1. Open the camera menu by clicking the  icon and set the Lock camera location to the "On" position. Press the "U" key to stop the script.

Estimating Forward Speed

1. In this lab you will be exploring sensor fusion techniques to localize the QBot Platform, beginning with estimating the forward speed of the QBot.
2. View the class definition for **QBPMovement()** in Section B within the [qbot_platform_functions](#) module. Scroll down to the function **diff_drive_forward_velocity_kinematics()** and complete the function by copying the relevant code section over from the **Forward Kinematics** lab in Skills Progression 1. This function calculates **fwdSpdOdom** and **turnSpdOdom**, which are the odometry-based estimates of forward and turn speeds, respectively.
3. Next, we will complete the code to estimate the forward speed using the accelerometer.

To drive the QBot using your keyboard, press and hold the space bar to arm the robot, and keep the space bar pressed as you drive the QBot using the movement keys. To stop the QBot, disarm it by releasing the space bar. The movement keys are as follows:

- a. Press the "A" and "D" keys to turn the QBot body
- b. Press the "I" and "K" keys to drive the QBot forward and back
- c. Optionally, press the "7" key to drive the QBot in a circle by applying a constant forward and turning speed command.

Run the code by clicking the play button  at the top right corner of the code. With Quanser Interactive Labs open, drive the QBot around while observing the X, Y, and Z signals on the Acceleration scope. Which direction corresponds to the forward movement direction? Stop the code by pressing the "U" key.

4. In Section C, modify the line **fwdAccel = 0** to set the variable **fwdAccel** to the forward acceleration component you identified in the previous step. This value is passed to the **accelIntegrator** generator, which accumulates a running sum of **fwdAccel*dt** at each loop to produce the integrated forward velocity.
5. Run the code but do not drive the QBot around. Instead, let the QBot remain stationary for at least one minute and observe the Forward Speed scope, which displays the individual estimates from each sensor. Then, drive the QBot forward and backward

using your keyboard keys. Release the spacebar to disarm the QBot and let it come to a stop.

6. What do you observe about the accelerometer-based forward speed estimate when the QBot is stationary? Take a screenshot to show your observations. Why does this happen? Stop the code.
7. In Section F, in the line **'Forward Speed Estimate': [fwdSpdAccel, 0, 0]**, replace the first **0** with **fwdSpdOdom**. This plots the odometry-based estimate on the Forward Speed scope. Repeat Step 5 while observing both signals on the scope. What do you observe about the odometry-based estimate? How does it compare to the accelerometer-based estimate? Take a screenshot to show your observations.
8. Stop your code. Now, we will correct the error in the accelerometer-based estimate using only proportional control. Right-click on the function **complementary_filter()** in Section B and view its definition. When this function is called in the main code, in Section C, the forward acceleration (**fwdAccel**) and the encoder-based speed estimate (**fwdSpdOdom**) are passed in as the **rate** and **correction** values. Here, the **error** is calculated as the difference between the generator **output** and the **correction** value passed into the generator. The error, multiplied by a proportional gain **Kp**, is subtracted from the **rate**. The result is integrated and becomes the output of the generator.
9. The line **fwdSpdFused = fwdSpeedComplementaryFilter.send((fwdAccel, fwdSpdOdom, 0.5, 0.0))** in Section C sends **fwdAccel** and **fwdSpdOdom** to the generator **fwdSpeedComplementaryFilter** with gains **Kp** set to 0.5 and **Ki** set to 0. In the line **'Forward Speed Estimate': [fwdSpdAccel, fwdSpdOdom, 0]** in Section F, replace the **0** with **fwdSpdFused** to plot the fused estimate on the Forward Speed Scope.
10. Run the code and drive the QBot around while observing the Forward Speed scope when the QBot is moving and idle. Stop the code.
11. Repeat step 10 as you set K_p to different values, including 1, 5, and 10. How does the fused estimate compare to the estimates from each individual sensor? How does increasing K_p affect the fused estimate?
12. Now, add integral control to complete the complementary filter: go to the function definition for **complementary_filter()**. Modify the line beginning with **temp =** to subtract the integrated error value multiplied by gain K_i . Save your changes.
13. In [sensor_fusion.py](#), change K_i to 0.05 and run the code. Let the QBot idle for a minute and then drive it forward and backward. Stop the code. Explain why the integral controller was added.

14. Run your code and observe the signals as you tune the gains K_p and K_i . How does each value affect how the signals are fused together? Tune the gains until the fused forward speed accurately tracks the correction signal with minimal noise. A sample tuned signal is shown in Figure 2.

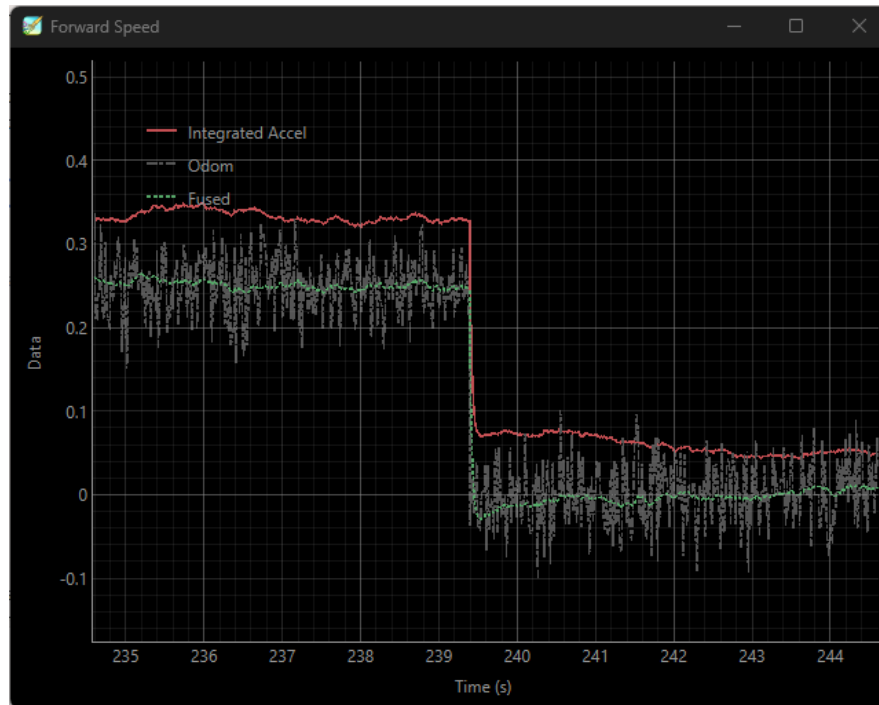


Figure 2: Fused forward speed estimate with tuned K_p and K_i gains

15. Complete the tuning for the two other instances of the complementary filter in Section C. Start with the **turnSpdComplementaryFilter**, which outputs the fused estimate of turn speed by fusing the rate of angular velocity about the z axis measured by the gyroscope, **gyroRate**, with an odometry-based estimate, **turnSpdOdorn**. Observe the Turn Speed scope and tune the K_p and K_i gains passed to **turnSpdComplementaryFilter**.
16. You now have fused estimates of the QBot's forward speed and turn speed. These are converted to speeds along the X and Y directions and combined into a vector [**x speed (m/s)**, **y speed (m/s)**, **ω (rad/s)**]. The **poseComplementaryFilter** uses this as the rate input and estimates the current position of the QBot using the position estimate from scan match as the correction input, which is the vector [**x position (m)**, **y position (m)**, **theta (rad)**].

Unlike the previous two filters, the K_p and K_i gains passed to **poseComplementaryFilter** are 3-element arrays. Each element sets the gain for the corresponding element of the pose vector. Tune each element using the X Position Estimates, Y Position Estimates, and Theta Estimates scopes to view the rate (integrated), correction, and fused estimates.

17. Run your code, drive the QBot around, and observe the estimated position on the Estimated Pose XY scope.

Comparison to Kalman Filter

18. Now, we will test an Extended Kalman Filter (EKF) for pose estimation and compare this sensor fusion technique to the complementary filter. The EKF takes the fused forward and turn speeds, `fwdSpdFused` and `turnSpdFused`, as control inputs. It uses them to predict the QBot's pose with a kinematic motion model, then corrects the prediction with the scan-match pose. The weight given to each correction is a gain calculated at every step to minimize the estimate's uncertainty, rather than set manually as in the complementary filter. This gain depends on the process noise covariance Q and the measurement noise covariance R , set by `Q_ekf` and `R_ekf` when `QBotEKF` is initialized in Section B. The completed implementation can be viewed inside the class definition for `QBotEKF()`.
19. In Section E, uncomment the line `poseEKF = ekf.x_hat[:,0]` to set `poseEKF` as the EKF position estimate. Run the code. Drive the QBot in a circle while observing the Pose Estimates Scope. How do the estimates from the EKF compare to the estimates from the complementary filter?
20. Stop the code when complete by pressing the "U" key. Ensure that you save a copy of your completed files for review later. Close Quanser Interactive Labs.